WebXR-Based Furniture Placement & Visualization Application

Real-Time Augmented Reality Furniture Placement Using Three.js and the WebXR Device API

Submitted by

Aditi S Pillai (2443001)

Drishya V Nair (2443022)

Under the guidance of

Dr. Balakrishnan

Table of Contents

1. Introduction

With Augmented Reality (AR), users get an opportunity to overlay digital content in real-time over the actual view of the world. There are many interesting possibilities when implementing augmented reality. For instance, interior design is one of the perfect use cases for AR technology, instead of visualizing how a certain item of furniture will look like in a room by looking at the picture of a photo or a floor plan, a person can add a life-size, scaled down 3D model right into his or her room.

This project describes implementation of mobile web-based AR Furniture Placement & Visualization app based on WebXR Device API and Three.js library. Contrary to AR furniture apps that are available on the market now, such as IKEA Place or Wayfair's View in Room, the application developed for this project does not require any native application installation from app stores. A user just needs to open a website link, provide camera access and start detecting the floor and adding furniture.

In order to get experience in developing web AR applications, this application was implemented from scratch using vanilla JavaScript and Three.js library as a WebGL engine for visualization purposes.

2. Objectives

- To design and develop a browser-based augmented reality application accessible without any app installation.
- To implement real-time horizontal surface (floor) detection using the WebXR Hit Test API.
- To enable placement of realistically scaled 3D furniture models onto detected surfaces.
- To provide intuitive touch-based interaction: selecting, dragging, rotating, and deleting placed furniture.
- To design a clean, categorized furniture-selection interface usable during an active AR session.
- To deploy the completed application to a publicly accessible, permanent HTTPS URL.

3. Literature Survey / Existing Systems

A number of commercial products offer furniture placement using AR technology, for example, IKEA Place, Houzz's View in My Room 3D, Wayfair's View in Room, and Amazon's View in Your Room. They all are proprietary native mobile apps which use AR frameworks of specific platforms

(ARKit on iOS and ARCore on Android) and have access to huge collections of products which are paid and can be purchased.

The main novelty of this project is in its deployment: instead of using the native application which works only on one particular platform and in one app store, this project utilizes WebXR Device API, a web standard for building immersive AR/VR experiences in the browser.

4. Technology Stack

Layer	Technology Used
3D Rendering Engine	Three.js (WebGL)
AR / Spatial Tracking	WebXR Device API — immersive-ar session, Hit Test API
Programming Language	Vanilla JavaScript (ES Modules)
3D Asset Format	glTF Binary (.glb) — Kenney Furniture Kit (CC0 license)
User Interface	HTML5 / CSS3, rendered as a WebXR DOM Overlay
Version Control	Git and GitHub
Hosting / Deployment	Vercel (static hosting with automatic HTTPS and CDN)
Development Testing	Chrome Remote DevTools with USB debugging / port forwarding

It is WebXR technology that was preferred over a native SDK, since it enables the use of the application in a normal mobile browser without the need for the app store to distribute it. It is three.js that was used since it is the most popular JavaScript 3D library that works with the WebXR session and the rendering pipeline. glTF/.glb was selected for the 3D assets since it is a compact web-friendly format optimised for real-time rendering.

5. System Architecture

The application follows a single-page, client-side-only architecture with no backend server or database, all logic runs in the browser.

5.1 System Flow

- The browser loads index.html, which sets up the canvas, the DOM overlay container, and imports the main JavaScript module.
- main.js initializes a Three.js scene, camera, and WebGL renderer, and requests a WebXR immersive-ar session with the hit-test and dom-overlay features.
- Once the session starts, a per-frame render loop performs hit-testing against the real world and updates a reticle indicating the detected floor position.
- A furniture catalogue (an array of model definitions grouped by category) drives a dynamically generated, tabbed selection UI in the DOM overlay.
- Touch events captured on the DOM overlay are interpreted to place, select, drag, rotate, or delete 3D furniture objects added to the Three.js scene.

5.2 Project Structure

Path	Purpose
index.html	Application entry point, canvas and DOM overlay markup, import map
src/main.js	Core application logic : AR session, hit-testing, furniture catalogue, gesture handling
styles/main.css	Styling for the DOM overlay UI (furniture picker, controls, buttons)
assets/models/	.glb furniture models grouped into Seating, Tables, and Decor

6. Features Implemented

- Real-time horizontal surface (floor) detection using WebXR Hit Testing, visualised with a tracking reticle.
- A categorized furniture catalogue (Seating, Tables, Decor) containing distinct 3D models.
- Automatically generated thumbnail previews for every furniture model, rendered offscreen from the actual 3D asset.
- Tap-to-place furniture at the reticle's detected position and orientation.
- Automatic real-world scale normalisation so every model renders at a realistic physical size.

- Tap-to-select placed furniture, with a visual highlight and a reliable padded hit-box for accurate touch detection.
- Single-finger drag-to-move for repositioning a selected object across the floor plane.
- A rotation slider (0°–360°) to orient a selected object.
- A delete control to remove the selected object.
- A “Clear All” button to remove every placed object all at once.
- A DOM-overlay based UI layered on top of the live AR camera feed, styled independently of the 3D scene.
- Deployment to a permanent, publicly accessible HTTPS URL via Vercel, connected to a GitHub repository for continuous deployment.

7. Implementation Details and Technical Challenges

7.1 Surface Detection

A ray is cast from the middle of the viewport of the device onto the real world in every frame of the animation using the XRHitTestSource class. When the ray hits a detected surface in the real world by the underlying AR framework (ARCore), the pose obtained is used to define the location and orientation of a reticle mesh in the form of a ring.

7.2 Automatic Scale Normalisation

The scale of each imported 3D model could be anything because they could have been made using some other internal scale system. In order to avoid manually adjusting the scale for each 3D model, the application calculates the bounding box of each 3D model after loading it and determines the scale factor so that the height of the model is the same as a realistic height in meters (For example 0.9m for a dining chair).

7.3 The DOM Overlay and Touch Handling Challenge

One major technical hurdle that was faced while developing this project was that when there is an active WebXR immersive-ar session, it consumes the usual touch events and turns the touches into 'select' events rather than feeding the raw touches to the webpage. This worked fine for the basic tap-and-place functionality, but it wasn't suitable for continuous gestures like dragging, as no touch-moves are detected using this method.

This issue was overcome by using the WebXR 'dom-overlay' capability, which made it possible for us to display a normal HTML overlay on top of our AR camera view and to detect real touch events on it. All the custom interactions, including the placing, selecting, dragging, and the in-screen buttons, were built based on the touch events detected by this overlay rather than the WebGL canvas itself.

7.4 Reliable Object Selection

Ray casting against the geometry of visible meshes on furniture objects was not an optimal approach for touch interaction because very thin elements (e.g., chair legs) could be hard to touch precisely with one's finger. The issue has been solved by adding invisible padded bounding box meshes to all placed objects, and these meshes are only used for selecting rays.

7.5 Material Cloning

Default cloning behavior for Three.js creates a duplicate of the hierarchy of the object being cloned but does not clone materials, thereby causing all copies of the same furniture object placed within the scene to use a single material. Consequently, when one object is highlighted, all copies of the model were highlighted as well due to the single material property. This was fixed by cloning the material whenever the object is placed.

7.6 Wall Detection (Descoped)

Wall detection using the WebXR 'plane detection' feature was attempted in an effort to make it possible to change wall colors. This idea was subsequently scrapped since tests showed that the capability is not uniformly supported by devices and the results are not consistent.

8. Testing

As WebXR sessions that use immersive-ar mode need a real camera and spatial tracking in the real world, there is no way to test it on a desktop environment or emulator. All testing was done on a physical Android smartphone using Chrome. The device was connected to a development computer through USB debugging, and the remote port forwarding in Chrome was used to deliver a local development build to the phone.

Testing covered the following scenarios:

- Surface detection accuracy across different floor textures and lighting conditions.
- Correct real-world scale and orientation of each placed furniture model.

- Reliable single-object selection when multiple items are placed close together.
- Smooth drag repositioning without unintended object duplication.
- Correct rotation behaviour via the on-screen slider.
- Deletion of individual objects and the 'Clear All' bulk-removal control.
- Category-tab switching and thumbnail-based furniture selection.
- End-to-end functionality on the final deployed HTTPS URL, independent of the local development setup.

9. Results and Screenshots

The application is now deployed successfully onto a publicly accessible URL, and it works as expected on a physical Android device – objects are detected as expected, the furniture is placed in proper scale in the real world, and all interactive functions work properly.

Deployed Application URL: https://webxr-interior-design-phi.vercel.app

(1) Floor detection with reticle

(2) Furniture category picker

(3) Placed furniture in a real room

(4) An object selected with rotate/delete controls visible

10. Limitations

- The application only functions in Chrome on Android, as WebXR's immersive-ar mode is not currently supported in Safari on iOS.
- Placed furniture layouts are not persisted; the scene resets when the session or page is closed.
- The furniture catalogue is a small, free asset set rather than a live, purchasable product catalogue.
- Wall/vertical surface detection and material recolouring were not implemented, due to inconsistent device-level support.
- Lighting and shadows use basic scene lighting rather than full environmental light estimation.

11. Future Scope

- Persisting room layouts using browser storage or a backend database, allowing users to save and revisit designs.
- Integrating a real, larger furniture catalogue with purchase links.

- Re-introducing wall detection with material/colour changing once broader device support is available.
- Adding environmental light estimation for more realistic shadow rendering.
- Building a complementary iOS experience using ARKit and USDZ Quick Look.
- Adding undo/redo functionality for placement actions.

12. Conclusion

This project has successfully demonstrated that a complete AR Interior Design experience that is fully functional and mobile can be created solely with the use of web technologies without the need for any native application. This was achieved through the creation of functionality such as surface detection, object scaling to match the size in the real world, and interaction controls through touch all directly against the WebXR Device API and Three.js, providing first-hand experience of the workings of AR that would normally be hidden behind proprietary software.

14. References

- WebXR Device API — W3C Immersive Web Working Group, https://www.w3.org/TR/webxr/
- WebXR Hit Test Module, https://www.w3.org/TR/webxr-hit-test-1/
- Three.js Documentation, https://threejs.org/docs/
- glTF 2.0 Specification, Khronos Group, https://www.khronos.org/gltf/
- Kenney Furniture Kit (CC0 asset pack), https://kenney.nl/assets/furniture-kit
- Vercel Documentation, https://vercel.com/docs